

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 20%

QUESTIONS: 15

Total Marks:25

Annexure A

Industry Problem Solving

Code of Conduct:

*I **THAMIZHARASAN S (192424087)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

THAMIZHARASAN S

Annexure A

Short Answer Test

1. A retail company wants to recommend products to customers based on past purchases. Design a solution using K-Nearest Neighbour or Case-Based Learning. Explain how similarity is measured, how recommendations are generated, and how the system adapts to new customer data.

ANSWER:

KNN can be used to recommend products based on the past purchases of similar customers.

- **Similarity Measurement:** Each customer is represented using a purchase vector, where 1 = purchased and 0 = not purchased. Euclidean distance is used to measure similarity. Smaller distance means more similar customers.
- **Find Neighbours:** The system finds the K nearest customers with similar purchase patterns.
- **Generate Recommendation:** Products purchased by the nearest customers but not purchased by the target customer are recommended.
- **Example:** If a customer purchased a laptop and mobile, and similar customers purchased headphones, the system recommends headphones.
- **Adaptation:** When a customer makes a new purchase, the purchase history is updated. KNN uses the updated data to generate new recommendations.

PYTHON CODE:

```
from sklearn.neighbors import NearestNeighbors
products = ["Laptop", "Mobile", "Headphone", "Shoes"]
data = [
    [1, 1, 0, 0],
    [1, 1, 1, 0],
    [0, 1, 1, 0],
    [0, 0, 1, 1],
    [1, 0, 1, 0]
]
knn = NearestNeighbors(n_neighbors=3, metric="euclidean")
knn.fit(data)
```

```

new_customer = [[1, 1, 0, 0]]
distance, neighbours = knn.kneighbors(new_customer)
recommendations = set()
for i in neighbours[0]:
    for j in range(len(products)):
        if data[i][j] == 1 and new_customer[0][j] == 0:
            recommendations.add(products[j])
print("Recommended Products:", recommendations)

```

OUTPUT:



```

from sklearn.neighbors import NearestNeighbors
products = ["Laptop", "Mobile", "Headphone", "Shoes"]
data = [
    [1, 1, 0, 0],
    [1, 1, 1, 0],
    [0, 1, 1, 0],
    [0, 0, 1, 1],
    [1, 0, 1, 0]
]
knn = NearestNeighbors(n_neighbors=3, metric="euclidean")
knn.fit(data)
new_customer = [[1, 1, 0, 0]]
distance, neighbours = knn.kneighbors(new_customer)
recommendations = set()
for i in neighbours[0]:
    for j in range(len(products)):
        if data[i][j] == 1 and new_customer[0][j] == 0:
            recommendations.add(products[j])
print("Recommended Products:", recommendations)

```

... Recommended Products: {'Headphone'}

Algorithm: K-Nearest Neighbour

Similarity: Euclidean Distance

Recommendation: Products bought by similar customers.

2. A manufacturing industry needs to predict machine output based on sensor readings that vary over time. Propose a solution using Locally Weighted Regression. Explain how local models improve prediction accuracy and how the system handles noisy or incomplete data.

ANSWER:

- Locally Weighted Regression (LWR) can be used to predict machine output from sensor readings such as temperature, pressure, and vibration.
- Local Model: For a new sensor reading, LWR gives higher weight to nearby training data and lower weight to distant data.
- Weight Calculation: The weight is calculated using distance. A common formula is:
$$w = \exp(-\text{distance}^2 / 2\tau^2)$$
- Prediction: A separate local regression model is created around the test point. This gives more accurate predictions when machine behaviour changes over time.
- Noisy Data: Noise can be reduced using smoothing, outlier removal, and suitable bandwidth (τ).
- Incomplete Data: Missing sensor values can be handled using mean/median imputation before applying LWR.

PYTHON CODE:

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([10, 20, 29, 41, 50])
x_test = np.array([[3.5]])
distance = X - x_test
weights = np.exp(-(distance ** 2) / 2)
model = LinearRegression()
model.fit(X, y, sample_weight=weights.ravel())
prediction = model.predict(x_test)
print("Predicted Machine Output:", prediction[0])
```

OUTPUT:

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([10, 20, 29, 41, 50])
x_test = np.array([[3.5]])
distance = X - x_test
weights = np.exp(-(distance ** 2) / 2)
model = LinearRegression()
model.fit(X, y, sample_weight=weights.ravel())
prediction = model.predict(x_test)
print("Predicted Machine Output:", prediction[0])
```

... Predicted Machine Output: 35.01815612431228

Algorithm: Locally Weighted Regression

Input: Sensor readings

Output: Predicted machine output

Main idea: Nearby data gets higher importance.

3. A healthcare company aims to classify patients into risk categories using historical medical data. Suggest a model using KNN or RBF networks. Explain how the algorithm handles large datasets, feature selection, and accuracy improvement in real-world scenarios.

ANSWER:

K-Nearest Neighbour (KNN) can be used to classify patients into Low, Medium, or High-risk categories using historical medical data.

- **Input Features:** Use features such as age, blood pressure, glucose level, cholesterol, etc.
- **Similarity Measurement:** Calculate the Euclidean distance between a new patient and existing patients. Smaller distance means greater similarity.
- **Classification:** Select the K nearest patients and assign the risk category based on majority voting.
- **Large Datasets:** For large datasets, efficient neighbour-search methods such as KD-Tree or Ball Tree can reduce search time.

- Accuracy Improvement: Normalize features, select important features, and choose the best value of K using validation. New patient records can be added to update the system.

PYTHON CODE:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
X = [
    [25, 110, 80],
    [30, 120, 90],
    [55, 150, 130],
    [60, 160, 140],
    [45, 140, 120]
]
y = ["Low", "Low", "High", "High", "Medium"]
scaler = StandardScaler()
X = scaler.fit_transform(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)
patient = scaler.transform([[50, 145, 125]])
print("Risk Category:", model.predict(patient)[0])
```

OUTPUT:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
X = [
    [25, 110, 80],
    [30, 120, 90],
    [55, 150, 130],
    [60, 160, 140],
    [45, 140, 120]
]
y = ["Low", "Low", "High", "High", "Medium"]
scaler = StandardScaler()
X = scaler.fit_transform(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)
patient = scaler.transform([[50, 145, 125]])
print("Risk Category:", model.predict(patient)[0])

... Risk Category: High
```

Algorithm: K-Nearest Neighbour

Similarity: Euclidean Distance

Classification: Majority voting of nearest patients

Accuracy: Improved using feature scaling and proper K selection.

4. An e-commerce platform wants to detect similar products for dynamic pricing strategies. Design a solution using Case-Based Learning. Discuss how similarity metrics are defined, how past cases are reused, and how the system scales with increasing product data.

ANSWER:

Case-Based Learning (CBL) can be used to find similar products and support dynamic pricing.

- Case Representation: Store each product as a case containing features such as price, category, brand, rating, and specifications.
- Similarity Measurement: Compare a new product with previous products using Euclidean distance or cosine similarity. Higher similarity means the products are more alike.
- Reuse of Past Cases: The system retrieves similar products and uses their previous prices and sales information to suggest a suitable price.
- New Cases: When a new product is sold, its details and sales results are stored as a new case for future recommendations.
- Scalability: For a large number of products, indexing and efficient similarity-search techniques can make case retrieval faster.

PYTHON CODE:

```
from sklearn.metrics.pairwise import cosine_similarity
products = [
    [20000, 4.2, 64],
    [25000, 4.5, 128],
    [30000, 4.6, 128],
    [18000, 4.0, 64]
]
new_product = [[24000, 4.4, 128]]
similarity = cosine_similarity(new_product, products)
```

```
best = similarity.argmax()
print("Most Similar Product:", best + 1)
print("Similarity:", round(similarity[0][best], 2))
```

OUTPUT:

```
from sklearn.metrics.pairwise import cosine_similarity
products = [
    [20000, 4.2, 64],
    [25000, 4.5, 128],
    [30000, 4.6, 128],
    [18000, 4.0, 64]
]
new_product = [[24000, 4.4, 128]]
similarity = cosine_similarity(new_product, products)
best = similarity.argmax()
print("Most Similar Product:", best + 1)
print("Similarity:", round(similarity[0][best], 2))
```

```
... Most Similar Product: 2
    Similarity: 1.0
```

Algorithm: Case-Based Learning

Similarity: Cosine Similarity

Application: Find similar products and reuse their past pricing information.

5. A banking system wants to detect potential loan defaulters based on customer financial history and transaction patterns. Design a solution using K-Nearest Neighbour or Logistic Regression. Explain how similarity or decision boundaries are used, how features are selected, and how the system adapts to new customer data over time.

ANSWER:

Logistic Regression can be used to predict whether a customer is likely to default on a loan.

- **Features:** Use important features such as income, loan amount, credit score, transaction history, and previous loan defaults.
- **Decision Boundary:** Logistic Regression calculates the probability of default. If the probability is greater than 0.5, the customer can be classified as a potential defaulter; otherwise, they are classified as low risk.

- Feature Selection: Unimportant or highly correlated features can be removed. Important features can be selected using correlation analysis or feature-selection techniques.
- Prediction: The trained model uses the customer's financial information to predict Default or No Default.
- Adapting to New Data: As new customer transactions and loan repayment records become available, the model can be periodically retrained with the updated dataset.

PYTHON CODE:

```
from sklearn.linear_model import LogisticRegression
X = [
    [50000, 10000, 750],
    [30000, 20000, 600],
    [70000, 15000, 800],
    [25000, 25000, 550],
    [60000, 12000, 780]
]
y = [0, 1, 0, 1, 0]
model = LogisticRegression()
model.fit(X, y)
customer = [[35000, 18000, 620]]
prediction = model.predict(customer)
probability = model.predict_proba(customer)[0][1]
if prediction[0] == 1:
    print("Potential Loan Defaulter")
else:
    print("Low Default Risk")
print("Default Probability:", round(probability, 2))
```

OUTPUT:

```
▶ from sklearn.linear_model import LogisticRegression
X = [
    [50000, 10000, 750],
    [30000, 20000, 600],
    [70000, 15000, 800],
    [25000, 25000, 550],
    [60000, 12000, 780]
]
y = [0, 1, 0, 1, 0]
model = LogisticRegression()
model.fit(X, y)
customer = [[35000, 18000, 620]]
prediction = model.predict(customer)
probability = model.predict_proba(customer)[0][1]
if prediction[0] == 1:
    print("Potential Loan Defaulter")
else:
    print("Low Default Risk")
print("Default Probability:", round(probability, 2))

... Potential Loan Defaulter
Default Probability: 1.0
```

Algorithm: Logistic Regression

Decision: Probability-based decision boundary

Output: Default / No Default

Adaptation: Retrain the model periodically using new financial and transaction data.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand